ErrorOr & Scalar — Ekip Sunumu

📁 Proje Yapısı (Clean Architecture)

```
deneme/
├── Domain/                    → Entity'ler
│   └── Entities/User.cs
├── Application/               → İş mantığı + ErrorOr
│   ├── Common/Errors/         → Merkezi hata tanımları
│   │   └── UserErrors.cs
│   └── Users/
│       ├── UserService.cs     → ErrorOr<T> dönen servis
│       └── CreateUserRequest.cs
└── deneme/ (WebAPI)           → Controller + Scalar
    ├── Controllers/
    │   ├── ApiController.cs   → Base controller (hata yönetimi)
    │   └── UsersController.cs → CRUD endpoint'leri
    └── Program.cs             → Servis kayıtları + Scalar config
```

1⃣ ErrorOr Nedir?

Exception fırlatmak yerine, hataları return value olarak döndürür.

❌ Geleneksel yaklaşım — Exception

```
public User GetUser(Guid id)
{
    var user = _users.Find(u => u.Id == id);
    if (user is null)
        throw new NotFoundException("Kullanıcı bulunamadı"); // ❌ Try-catch'e mahkum
    return user;
}
```

✅ ErrorOr yaklaşımı — Değer olarak hata

```
public ErrorOr<User> GetUser(Guid id)
{
    var user = _users.FirstOrDefault(u => u.Id == id);
    if (user is null)
        return UserErrors.NotFound(id);  // Hata bir değer, exception değil

    return user;                         // Başarı durumunda doğrudan döner
}
```

> [!IMPORTANT] Dönüş tipi `ErrorOr<T>` — ya başarılı sonucu ya da hata listesini taşır. Exception yok, try-catch yok!

2⃣ Hataları Merkezi Tanımlama

Hatalar `Application/Common/Errors/` altında tek yerde tanımlanır:

```
public static class UserErrors
{
    public static Error NotFound(Guid id) =>
        Error.NotFound("User.NotFound", $"'{id}' ID'li kullanıcı bulunamadı.");

    public static Error DuplicateEmail(string email) =>
        Error.Conflict("User.DuplicateEmail", $"'{email}' e-posta adresi zaten kullanımda.");

    public static Error InvalidName =>
        Error.Validation("User.InvalidName", "Kullanıcı adı boş olamaz.");
}
```

Hazır hata tipleri: `Validation` · `NotFound` · `Conflict` · `Unauthorized` · `Failure` · `Unexpected`

3⃣ Service Katmanında Kullanım

```
public ErrorOr<User> CreateUser(string name, string email)
{
    if (string.IsNullOrWhiteSpace(name))
        return UserErrors.InvalidName;           // ← Validation hatası

    if (_users.Any(u => u.Email == email))
        return UserErrors.DuplicateEmail(email);  // ← Conflict hatası

    var user = new User { Id = Guid.NewGuid(), Name = name, Email = email };
    _users.Add(user);
    return user;                                  // ← Başarı
}
```

4⃣ Controller'da Match Pattern

Base ApiController — Hata → HTTP Status Code dönüşümü

```
public class ApiController : ControllerBase
{
    protected IActionResult Problem(List<Error> errors)
    {
        var firstError = errors.First();
        var statusCode = firstError.Type switch
        {
            ErrorType.NotFound    => 404,
            ErrorType.Validation  => 400,
            ErrorType.Conflict    => 409,
            ErrorType.Unauthorized => 403,
            _                     => 500
        };
        return Problem(title: firstError.Description, statusCode: statusCode);
    }
}
```

UsersController — Temiz ve sade

Search ⌃K Ask AI

Introduction

Users

/api/Users POST

/api/Users GET

/api/Users/{id} GET

/api/Users/{id} DEL

Models

Generate MCP

/api/Users

Body · CreateUserRequest required application/json

email string required

name string required

Responses

200 OK

POST /api/Users Shell Curl

```
curl http://localhost:5147/api/Users \
  --request POST \
  --header 'Content-Type: application/json' \
  --data '{
  "name": "",
  "email": ""
}'
```

Ask AI Agent ▶ Test Request

200

No Body

OK

POST http://localhost:5147 /api/Users ▶ Send ↗ Open API Client

All

Cookies

Key	Value

Headers

✓	accept	*/*
✓	content-type	application/json
	Key	Value

Query Parameters

Key	Value

Request Body

JSON

```
{
  "name": "tunahan",
  "email": "tunahan@gmail.com"
}
```

Code Snippet

100ms 201 Created

All

Cookies

Request Headers

accept	*/*
content-type	application/json

Response Headers

Content-Type	application/json; charset=utf-8
Date	Sun, 26 Apr 2026 22:09:30 GMT
Location	/api/users/9b455d3e-8245-4ee5-b811-c584ce056a40
Server	Kestrel
Transfer-Encoding	chunked

Body Download

application/json

```
{
  "id": "9b455d3e-8245-4ee5-b811-c584ce056a40",
  "name": "tunahan",
  "email": "tunahan@gmail.com",
  "createdAt": "2026-04-26T22:09:31.1323579Z"
}
```

Search ⌃K Ask AI

Introdu

Users

/api/

/api/

/api/

/api/

Models

POST http://localhost:5147 /api/Users ▶ Send ↗ Open API Client

All

Cookies

Key	Value

Headers

✓	accept	*/*
✓	content-type	application/json
	Key	Value

Query Parameters

Key	Value

Request Body

JSON

```
{
  "name": "",
  "email": "tunahan@gmail.com"
}
```

Code Snippet

8ms 400 Bad Request

All

Cookies

Request Headers 2

Response Headers 4

Body Download

application/problem+json

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "User.InvalidName": [
      "Kullanıcı adı boş olamaz."
    ]
  },
  "traceId": "00-ba2d75c8dd527da23bf3566fec239236-089234a9935ab92d-00"
}
```

```
public class UsersController(UserService userService) : ApiController
{
    [HttpPost]
    public IActionResult Create(CreateUserRequest request)
    {
        var result = userService.CreateUser(request.Name, request.Email);
        return result.Match(
            user => Created($"/api/users/{user.Id}", user),
            Problem   // ← Base controller'a delege
        );
    }

    [HttpGet("{id:guid}")]
    public IActionResult Get(Guid id)
    {
        var result = userService.GetUser(id);
        return result.Match(Ok, Problem);  // ← Tek satır!
    }
}
```

> [!TIP] `result.Match(Ok, Problem)` — başarıysa `Ok()`, hatalıysa `Problem()`. Try-catch yok, temiz kod!

5⃣ Scalar Nedir?

Swagger UI'ın modern, hızlı ve güzel alternatifi.

Kurulum (Program.cs):

```
builder.Services.AddOpenApi();

app.MapOpenApi();
app.MapScalarApiReference(options =>
{
    options
        .WithTitle("ErrorOr & Scalar Demo API")
        .WithTheme(ScalarTheme.DeepSpace);
});
```

Erişim:

```
http://localhost:5147/scalar/v1
```

Özellik	Swagger UI	Scalar
Arayüz	Eski, klasik	Modern, şık
Tema desteği	⃣	⃣ Dark mode + temalar
API test	Kısıtlı	⃣ Built-in (Postman gibi)
Hız	Yavaş	⃣ Çok hızlı
Kurulum	Birden fazla paket	⃣ Tek paket

⃣ Canlı Demo

```
dotnet run --project deneme
```

Sonra tarayıcıda → http://localhost:5147/scalar/v1

Test senaryoları:

#	İşlem	Beklenen
1	`POST /api/users` `{"name":"Ali","email":"ali@test.com"}`	201 Created
2	`POST /api/users` `{"name":"","email":"x@test.com"}`	400 — Ad boş olamaz
3	`POST /api/users` `{"name":"Veli","email":"ali@test.com"}`	409 — E-posta zaten kullanımda
4	`GET /api/users/{random-guid}`	404 — Kullanıcı bulunamadı
5	`GET /api/users`	200 — Liste

Özet

Kütüphane	Ne yapar	Neden kullanmalıyız
ErrorOr	Hata yönetimini değer bazlı yapar	Try-catch kirliliğini yok eder, tip güvenli
Scalar	API dökümantasyonu sunar	Swagger'dan modern, hızlı ve güzel